

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение

высшего образования

«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ Кафедра ИБК

К ЗАЩИТЕ ДОПУСТИТЬ

Зав. кафедрой

_____/_____/_____
подпись инициалы, фамилия

«____» _____ 20__ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Тема Программное обеспечение для управления камерой и обработки данных в системе
инфракрасного мониторинга температуры объекта

Обучающийся _____/_____
подпись инициалы, фамилия

Обозначение работы ВКР-УЛГТУ-09.03.02-22/2410-2026 Группа ИСТбд-41

Направление подготовки 09.03.02 «Информационные системы и технологии»
код, наименование

Руководитель работы _____/_____
подпись инициалы, фамилия

Консультанты:

Экономический раздел _____/_____
наименование раздела подпись, дата инициалы, фамилия

Ульяновск, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение

высшего образования

«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ Кафедра ИВК

Направление подготовки информационные системы и технологии

УТВЕРЖДАЮ

Зав. кафедрой

_____/_____/_____
подпись инициалы, фамилия

«____» _____ 20__ г.

ЗАДАНИЕ

на выпускную квалификационную работу

обучающемуся Лапманову Николаю Николаевичу курса 4 группы ИСТбд-41
фамилия, имя, отчество

1. Тема работы Программное обеспечение для управления камерой и обработки данных в системе инфракрасного мониторинга температуры объекта

утверждена приказом по университету № 64 от «15» _____ января _____ 2026 г.

2. Срок сдачи обучающимся законченной работы «23» _____ июня _____ 2026 г.

3. Исходные данные к работе Создать программное обеспечение для управления камерой и обработки данных в системе инфракрасного мониторинга температуры объекта

4. Содержание пояснительной записки техническое задание на создание системы; информационное, алгоритмическое, программное обеспечение системы; тестирование системы.

5. Перечень графического материала (чертежей) схема алгоритма компиляции программного кода примеров (чертёж формата А1 по ГОСТ 19.701-90), концептуальная схема базы данных (чертёж формата А1 в нотации IDEF1X).

6. Календарный график работы на весь период (с указанием сроков выполнения и содержания отдельных этапов)

№ этапа	Содержание этапа	Срок выполнения
1	Реализация предварительной версии системы и её показ на выставке программных продуктов недели студенческой науки	14.04.2026
2	Реализация окончательной версии системы	31.05.2026
3	Тестирование системы	02.06.2026
4	Написание пояснительной записки	04.06.2026
5	Создание чертежей	06.06.2026
6	Прохождение нормоконтроля	09.06.2026
7	Защита	23.06.2026

7. Консультанты

Раздел	Ф.И.О. консультанта	Подпись, дата	
		задание выдал	задание принял
Экономический раздел	М.В. Рыбкина		

8. Дата выдачи задания «17» марта 20 26 г.

Руководитель доцент каф. ИВК, к.т.н., доцент / М. В. Тамьярова /
должность, учёная степень, учёное звание подпись инициалы, фамилия

Задание принял к исполнению / Н. Н. Лащманов /
подпись инициалы, фамилия

АННОТАЦИЯ

Выпускная квалификационная работа: Лашманова Николая Николаевича по теме *«Программное обеспечение для управления камерой и обработки данных в системе инфракрасного мониторинга температуры объекта»*.

Руководитель Тамьярова Майя Владиславовна. Работа защищена на кафедре «Измерительно-вычислительные комплексы» УлГТУ в 2026 году.

Пояснительная записка: 67 страниц, 6 разделов, 1 приложение, 9 рисунков, 19 таблиц, 21 источник.

Ключевые слова: измерение температуры, датчики, Python, MySQL, микроконтроллеры.

Разработанное программное обеспечение предназначено для автоматизации процесса инфракрасного мониторинга температуры электронных компонентов, печатных плат и готовых изделий в лабораторных и цеховых условиях. Подсистема обеспечивает приём данных от модуля ESP32-CAM и от подсистемы управления поворотной платформой с датчиком MLX90614. Выполняется автоматическое наложение на оптическое изображение тепловой карты с визуализацией температурных точек в цветовом градиенте. Результаты измерений сохраняются в реляционную базу данных MySQL, а также экспортируются в форматы CSV и DOCX. Программное обеспечение реализовано на языке Python с использованием библиотек Tkinter, Pillow и SciPy. Система является кроссплатформенной, имеет открытый исходный код и может быть использована в учебных и исследовательских целях, а также для интеграции в комплексные системы теплового контроля.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ	8
ВВЕДЕНИЕ	9
1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА СОЗДАНИЕ СИСТЕМЫ	12
1.1 Назначение системы в целом	12
1.2 Характеристика объекта автоматизации	13
1.2.1 Общее описание	13
1.3 Общие требования к подсистеме	14
1.3.1 Требования к структуре и функционированию	14
1.3.2 Дополнительные требования	15
1.4 Требования к функциям, выполняемым системой	16
1.4.1 Функция подключения к камере	16
1.4.2 Функция визуализации данных	16
1.4.3 Функция экспорта в CSV	16
1.4.4 Функция создания отчета в DOCX	17
1.5 Требования к видам обеспечения	17
1.5.1 Требования к информационному обеспечению	17
1.5.2 Требования к алгоритмическому обеспечению	17
1.5.3 Требования к программному обеспечению	18
1.6 Анализ аналогичных программных продуктов	18
2 ИНФОРМАЦИОННОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ	21
2.1 Выбор средств управления данными	21
2.2 Проектирование базы данных	22
2.2.1 Концептуальная схема базы данных	22
2.2.2 Внутренняя схема базы данных	24

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ				
Изм.	Лист	№ докум.	Подпись	Дата					
Разраб.		Лашманов Н. Н.			Программное обеспечение для управления камерой и обработки данных в системе инфракрасного мониторинга температуры объекта	Лит.	Лист	Листов	
Провер.		Тамьярова М. В.				У	5	67	
Консульт.						ИСТДД — 41			
Н. Контр.		Шикина В. Е.							
Утв.									

2.3 Проектирование файлов данных	25
2.4 Организация сбора, передачи, обработки и выдачи информации	26
3 АЛГОРИТМИЧЕСКОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ	27
3.1 Алгоритм наложения температурной информации на изображение и сохранения в базу данных	27
3.2 Алгоритм экспорта в файл в формате CSV	27
3.3 Алгоритм экспорта в файл в формате DOCX	27
4 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ	32
4.1 Структура программного обеспечения и функции его компонентов.....	32
4.2 Выбор компонентов программного обеспечения	32
4.2.1 Операционная система.....	32
4.2.2 Инструментальное средство разработки и язык программирования ...	33
4.2.3 Вспомогательное программное обеспечение	33
4.3 Разработка прикладного программного обеспечения	34
4.3.1 Структура прикладного программного обеспечения	34
4.4 Особенности реализации, эксплуатации и сопровождения подсистемы	36
4.5 Руководство пользователя.....	37
4.5.1 Требования к условиям эксплуатации	37
4.5.2 Инсталляция и настройка	38
4.5.3 Порядок и особенности работы	38
4.5.4 Исключительные ситуации и их обработка	41
5 ТЕСТИРОВАНИЕ СИСТЕМЫ.....	42
5.1 Условия и порядок тестирования	42
5.2 Исходные данные для контрольных примеров.....	42
5.2.1 Описание ситуаций для элементов системы	42
5.3 Результаты тестирования	43
6 ЭКОНОМИЧЕСКИЙ РАЗДЕЛ	44
6.1 Расчёт показателей трудоёмкости для программного продукта.....	44

6.2 Расчёт затрат на материальные ресурсы.....	45
6.3 Расчет затрат на разработку системы.....	46
6.4 Расчет затрат на оплату труда.....	47
6.5 Расчет отчислений на социальные нужды.....	48
6.6 Себестоимость проекта.....	48
6.7 Расчет плановой прибыли	49
6.8 Определение экономической эффективности проекта	49
6.9 Выводы по технико-экономическому анализу и обоснованию проекта разработки.....	50
ЗАКЛЮЧЕНИЕ	51
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	53
ПРИЛОЖЕНИЕ А	56

СПИСОК ИСПОЛЬЗОВАННЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

БД (база данных) – это упорядоченный набор структурированной информации или данных, которые обычно хранятся в электронном виде в компьютерной системе.

ГОСТ (государственный стандарт) – это государственный стандарт, который включает в себя требования государства к качеству продукции, его геометрические размеры, отклонения от эталона и т. д.

РД (руководящий документ) – нормативный документ, содержащий правила, общие принципы, характеристики объектов нормирования, касающиеся определенных видов деятельности или их результатов, доступный широкому кругу потребителей (пользователей) и утвержденный в установленном в отрасли порядке.

СПО (свободное программное обеспечение) – программное обеспечение, пользователи которого имеют права на его неограниченную установку, запуск, свободное использование, изучение, распространение и изменение, а также распространение копий и результатов изменения.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		8

ВВЕДЕНИЕ

Современное приборостроение и радиоэлектронная промышленность предъявляют высокие требования к качеству и надёжности выпускаемых изделий. Одним из ключевых параметров, влияющих на работоспособность электронных компонентов и печатных плат, является их температурный режим. Перегрев элементов может привести к деградации характеристик, внезапным отказам и сокращению срока службы оборудования. В связи с этим возникает необходимость в оперативном и достоверном тепловом контроле как на этапе производства, так и в процессе испытаний готовых изделий.

Традиционно для выявления перегретых участков используются ручные тепловизоры или точечные пирометры. Однако такие методы обладают рядом недостатков: они требуют постоянного участия оператора, не позволяют автоматизировать процесс измерения в заданных контрольных точках, а также не предоставляют встроенных средств для хранения и последующего анализа полученных данных. В результате снижается производительность контроля, возрастает вероятность ошибок из-за человеческого фактора, а накопление результатов испытаний затруднено.

Инфракрасный мониторинг с применением автоматизированной поворотной платформы и недорогих датчиков (например, MLX90614) позволяет решить перечисленные проблемы. Однако для полноценного использования такого оборудования требуется специализированное программное обеспечение, которое обеспечивало бы управление камерой, циклический обход заданных точек, приём и маркировку тепловизионных снимков, сохранение результатов в структурированном виде, а также их экспорт для последующего анализа.

Целью данной работы является разработка программного обеспечения для управления камерой и обработки данных в системе инфракрасного

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		9

мониторинга температуры объекта, позволяющего автоматизировать процесс теплового контроля электронных компонентов, печатных плат и готовых изделий в лабораторных и цеховых условиях.

Благодаря внедрению разработанного ПО оператор получает возможность автоматического циклического измерения температуры в заданных точках, сохранения каждого измерения вместе с фотографией, на которой визуально отмечены положение точки и измеренное значение. Формирование отчётов и экспорт в CSV выполняются автоматически, что упрощает документооборот и снижает трудоёмкость контроля.

Более подробно особенности проекта рассматриваются в основной части пояснительной записки. Техническое задание содержит требования к структуре системы, функциям и видам обеспечения. В информационном обеспечении описываются форматы входных и выходных данных, организация хранения изображений и результатов измерений в файловой системе. Алгоритмическое обеспечение включает описание основных алгоритмов обработки изображений и приёма данных. Программное обеспечение раскрывает структуру приложения, выбор компонентов и руководство пользователя. Результаты тестирования подтверждают соответствие системы заявленным требованиям. В экономическом разделе приведены расчёты затрат и эффективности.

При написании выпускной квалификационной работы использовались следующие источники:

Источник [11] – содержит основные методы обработки и наложения графических элементов на изображения, что было применено при реализации подсистемы маркировки кадров.

Работа с библиотекой Pillow подробно освещена в официальной документации [3], что позволило эффективно реализовать декодирование JPEG, рисование текста и координатных маркеров.

В источнике [12] рассматриваются основы создания графических интерфейсов на Tkinter, а также работа с файловой системой и CSV-файлами, что легло в основу модуля экспорта и пользовательского интерфейса.

Официальная документация по Python [6] и руководство по работе с сетевыми сокетами использовались для организации приёма данных от внешнего модуля ESP32-CAM по протоколу HTTP.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		11

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА СОЗДАНИЕ СИСТЕМЫ

1.1 Назначение системы в целом

Разрабатываемая система предназначена для автоматизированного бесконтактного теплового контроля электронных компонентов, печатных плат и готовых изделий в лабораторных и цеховых условиях приборостроительного предприятия. Контроль осуществляется с применением инфракрасного датчика температуры MLX90614, закреплённого на двухосевой поворотной платформе.

Система состоит из двух взаимодействующих подсистем, разрабатываемых в рамках двух выпускных квалификационных работ.

Разрабатываемая подсистема предназначена для сбора, сохранения, визуализации и представления данных, полученных с подсистемы в рамках другой работы ВКР и модуля камеры.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		12

1.2 Характеристика объекта автоматизации

1.2.1 Общее описание

Объектом автоматизации является процесс сбора, анализа и обработки данных, поступающих с установки. Подсистема принимает температурные показатели, координаты и изображение с установки для записи в базу данных.

Система решает следующие задачи:

а) Сбор, хранение и визуализацию результатов измерений: после цикла работы установки, данные измерений записываются в базу данных системы для последующего сохранения, анализа и обработки. Визуализация производится благодаря фотосъёмке с модуля камеры ESP32-CAM и рисованию точек с цветами, зависящими от температуры. Градиент от красного, выражающего температуру выше двойного среднего рассчитанного значения, до тёмно-синего, выражающего температуру ниже нуля. Платы с компонентами одной партии или производства могут иметь одинаковые дефекты, которые можно увидеть на фото.

б) Формирование отчётов и экспорт данных: после сохранения данных система формирует отчёт с полученными значениями для ускоренного анализа результатов испытаний. В отчёт пользователем заносятся название изделия прошедшего испытания, дата и время. Основные точки, на которые следует обратить внимание выделяются и отображаются в отчёте.

в) Выдачу управляющих сигналов для внешних исполнительных устройств (вентилятор, нагреватель, сигнализация) на основе результатов нечёткого анализа: в случае резкого превышения показателями заданных норм, срабатывает сигнализация в виде мигающего красного светодиода. Данное оповещение будет сигнализировать о поломке прибора, превышения

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист 13
Изм.	Лист	№ докум.	Подпись	Дата		

тока питания или иных ситуациях, во время которых может резко превышать температура.

1.3 Общие требования к подсистеме

1.3.1 Требования к структуре и функционированию

Подсистема должна представлять собой программно-аппаратный комплекс для визуализации и регистрации температурного состояния объекта с использованием тепловой карты и последующего формирования отчётов.

В состав подсистемы должны входить следующие модули:

– Модуль сбора данных: получает изображение и температурные данные с камеры и взаимодействующей подсистемы в рамках другой ВКР соответственно.

– Модуль визуализации и разметки: накладывает на изображение информацию о температуре в виде точек.

– Модуль хранения: записывает каждое измерение в реляционную базу данных, изображение сохраняется в отдельную папку.

– Модуль экспорта: выгружает выбранный диапазон записей в CSV-файл.

– Модуль создания отчёта: создаёт отчёт на основе полученных данных в документ в формате DOCX.

Система должна состоять из двух частей: ПК для приёма данных и управления системой и модуль с камерой для получения изображений.

Перспективы развития системы предполагают расширение базового функционала путём добавления дополнительных датчиков.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		14

1.3.2 Дополнительные требования

1.3.2.1 Требования к персоналу

Система должна быть однопользовательская.

Пользователь должен владеть навыками пользования ПК на базовом уровне: уметь работать с *.docx и *.csv файлами и изображениями.

1.3.2.2 Требования к надежности

Система должна сохранять работоспособность и обеспечивать восстановление своих функций при возникновении следующих внештатных ситуаций:

- при сбоях в системе электроснабжения аппаратной части, приводящих к перезагрузке ОС, восстановление программы должно происходить после перезапуска ОС и запуска исполняемого файла системы;
- при ошибках в работе аппаратных средств (кроме носителей данных и программ) восстановление функции системы возлагается на ОС;
- при ошибках, связанных с программным обеспечением (ОС и драйверы устройств), восстановление работоспособности возлагается на ОС.

В системе должна быть обеспечена возможность восстановления данных с внешнего накопителя после восстановления активного накопителя. БД системы необходимо резервировать минимум 1 раз в месяц.

Специальные дополнительные требования по составу и количественным значениям показателей надежности для подсистем модернизируемых модулей и, соответственно, к создаваемой системе в целом не предъявляются.

1.4 Требования к функциям, выполняемым системой

1.4.1 Функция подключения к камере

Данная функция подключается к камере модуля ESP32-CAM и получает данные.

Входные данные: IP-адрес модуля.

Выходные данные: статус подключения.

Сроки: первая очередь.

1.4.2 Функция визуализации данных

Данная функция откладывает точки с температурными данными на изображении.

Входные данные: изображение, данные о температуре, координаты точек.

Выходные данные: промаркированное изображение.

Требование: обработка одного кадра (включая декодирование JPEG, рисование, сохранение в буфер) должна занимать не более 0,2 с.

Сроки: первая очередь.

1.4.3 Функция экспорта в CSV

Входные данные: температура, координаты точек, временная метка.

Выходные данные: файл *.csv с данными температуры.

Требование: экспорт 1000 записей должен выполняться не дольше 5 с.

Сроки: вторая очередь.

1.4.4 Функция создания отчета в DOCX

Входные данные: промаркированное изображение, максимальная температура, минимальная температура.

Выходные данные: файл *.docx в качестве отчета с промаркированным в результате работы системы изображением, максимальной и минимальной температурой.

Сроки: третья очередь.

1.5 Требования к видам обеспечения

1.5.1 Требования к информационному обеспечению

Реляционная СУБД должна использоваться для хранения данных системы. Сбор данных для приложения должен быть автоматизирован. Доступ к данным является однопользовательским. Время выполнения запросов должно обеспечивать достаточное быстродействие для работы приложения.

Время получения изображения не должно превышать 10 секунд.

Необходимо использовать резервное копирование базы данных раз в месяц.

Защита данных не требуется, так как в данном приложении не используется личная информация.

1.5.2 Требования к алгоритмическому обеспечению

Требования к алгоритмическому обеспечению не предъявляются.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист 17
Изм.	Лист	№ докум.	Подпись	Дата		

1.5.3 Требования к программному обеспечению

Система должна быть платформонезависимой. Структура ПО должна быть легко расширяема и масштабируема.

При разработке прикладного программного обеспечения необходимо использовать язык программирования Python 3.10. Для удобства разработки также необходимо использовать библиотеку Pillow для редактирования изображений.

Взаимодействие с системой должно осуществляться через графический интерфейс. Интерфейс не должен быть перенасыщен графическими элементами и должен предусматривать быстрое отображение в виде экранной формы. Навигационные элементы должны быть выполнены в удобной для пользователя форме.

1.6 Анализ аналогичных программных продуктов

Для оценки конкурентных преимуществ разрабатываемого ПО были проанализированы три программных инструмента, используемых в тепловизионной диагностике. В таблице 1.1 приведён сравнительный анализ разрабатываемой системы с конкурентными продуктами.

1. Fluke SmartView

Программа поставляется с тепловизорами Fluke и предназначена для просмотра и базового анализа ИК-изображений. Позволяет накладывать маркеры, изменять палитру, формировать отчёты.

Недостатки: не поддерживает кластеризацию; работа с данными возможна только в проприетарном формате Fluke; отсутствует регрессионный анализ временных рядов; нельзя напрямую импортировать данные от сторонних датчиков.

2. FLIR Tools

Аналогичный продукт для устройств FLIR. Помимо просмотра, позволяет корректировать параметры съёмки и готовить простые отчёты.

Недостатки: ориентирован исключительно на работу с камерами FLIR, закрытый формат хранения; аналитические функции ограничены вычислением минимума/максимума/среднего; отсутствуют модули кластеризации и прогнозирования.

3. ThermaCAM Researcher (FLIR)

Профессиональное ПО для углублённого анализа тепловизионных последовательностей, поддерживает построение температурных графиков во времени для выбранных точек.

Недостатки: высокая стоимость лицензии, закрытый исходный код, требует формата FLIR; встроенная статистика не включает автоматическую кластеризацию, а прогнозирование ограничено линейной аппроксимацией без расчёта прогноза на заданный горизонт.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		19

В таблице 1.1 представлен сравнительный анализ разрабатываемой системы и аналогов.

Таблица 1.1 – Сравнительный анализ

Критерий	Разрабатываемое ПО	Fluke SmartView	FLIR Tools	ThermaCAM Researcher
Приём данных от сторонних устройств	+	-	-	-
Хранение в открытой БД	+	-	-	-
Экспорт в CSV с изображениями	+	-	-	-
Создание отчётов в Microsoft Word	+	+	-	-
Скорость работы ПО	-	+	+	+
Точность измерений	-	-	+	+
Открытый исходный код	+	-	-	-

Разрабатываемое ПО выгодно отличается универсальностью приёма данных и открытостью, что делает его пригодным для учебных и исследовательских задач, а также для интеграции в комплексные системы мониторинга, несмотря на недостатки в виде меньшей скорости работы и точности измерений.

2 ИНФОРМАЦИОННОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ

2.1 Выбор средств управления данными

Для организации хранения результатов измерений в разрабатываемой подсистеме необходимо выбрать СУБД. Основные требования к хранению данных вытекают из технического задания:

- поддержка реляционной модели (данные структурированы, имеют чёткие типы и связи);
- работа в однопользовательском режиме (система разворачивается на одном компьютере оператора);
- высокая скорость выполнения операций вставки (не более 50 мс на запись) и выборки;
- поддержка стандартного языка SQL;
- простота резервного копирования и восстановления.

Были рассмотрены три варианта: SQLite, MySQL и PostgreSQL.

– SQLite – встраиваемая СУБД, не требует отдельного сервера, проста в настройке. Однако её производительность при одновременной записи и чтении может снижаться, а поддержка типов данных и конкурентного доступа ограничена. Для нашего сценария (один процесс записывает данные, другой – анализирует) SQLite подходит, но отсутствие гибких механизмов разграничения доступа и менее развитые средства для работы с большими текстовыми полями делают её менее предпочтительной.

– PostgreSQL – мощная объектно-реляционная СУБД с отличной масштабируемостью. Избыточна для данной задачи, требует дополнительных ресурсов и администрирования, что нецелесообразно для лабораторного/цехового применения.

– MySQL – широко распространённая реляционная СУБД, сочетающая производительность, надёжность и простоту администрирования. Поддерживает необходимые типы данных (TEXT, REAL, INTEGER), обеспечивает скорость вставки до 50 мс, имеет удобные средства резервного копирования (mysqldump). При этом MySQL может работать как на том же компьютере, так и на удалённом сервере, что даёт возможность будущего расширения.

На основании анализа выбран MySQL в качестве СУБД для хранения всех измерений.

2.2 Проектирование базы данных

2.2.1 Концептуальная схема базы данных

Модель «сущность-связь» представлена на рисунке 2.1

В таблице 2.1 представлен список сущностей БД. В таблицах 2.2-2.4 представлены атрибуты сущности БД.



Рисунок 2.1 – Модель «сущность-связь»

Таблица 2.1 – Сущности модели

№	Название сущности	Описание
1	Сессии	Хранятся временные метки сессий работы программы
2	Измерения	Данные отдельных измерений
3	Температурные точки	Данные координат и температур точек измерения

Таблица 2.2 – Атрибуты сущности «Сессии»

№	Название атрибута	Тип	Описание
1	Id	Числовой	Номер сессии
2	Время создания	Дата/время	Время создания сессии

Таблица 2.3 – Атрибуты сущности «Измерения»

№	Название атрибута	Тип	Описание
1	Id	Числовой	Номер измерения
2	Id сессии	Числовой	Номер сессии, к которой относится измерение
3	Временная метка	Дата/время	Время создания сеанса измерения
4	Путь до изображения	Текстовый	Путь до изображения в сеансе измерения

Таблица 2.4 – Атрибуты сущности «Температурные точки»

№	Название атрибута	Тип	Описание
1	Id	Числовой	Номер точки
2	Id измерения	Числовой	Номер сеанса измерения, в котором измерена точка
3	Позиция по X	Числовой	Координата измеренной точки на фотографии по оси X
4	Позиция по Y	Числовой	Координата измеренной точки на фотографии по оси Y
5	Температура	Числовой	Измеренная температура в точке

2.2.2 Внутренняя схема базы данных

В таблицах 2.5-2.7 представлены физические модели БД.

Таблица 2.5 – Физическая модель sessions [Сессии]

№	Название поля	Тип и размер	Значение по умолчанию	Допустимые значения
1	id [id]	INT	1	от 1 до 2147483647
2	created_at [Время создания]	DATETIME	-	00:00:00 12.01.2026

В таблице sessions в качестве индекса используется первичный ключ по полю id, который является уникальным и сортируется по возрастанию.

Таблица 2.6 – Физическая модель measurements [Измерения]

№	Название поля	Тип и размер	Значение по умолчанию	Допустимые значения
1	id [id]	INT	1	от 1 до 2147483647
2	session_id [Id сессии]	INT	1	от 1 до 2147483647
3	timestamp [Временная метка]	DATETIME	-	00:00:00 12.01.2026
4	image_path [Путь до изображения]	VARCHAR(255)	-	Без ограничений

В таблице measurements в качестве индекса используется первичный ключ по полю id, который является уникальным и сортируется по возрастанию.

Таблица 2.7 – Физическая модель temperature_points [Температурные точки]

№	Название поля	Тип и размер	Значение по умолчанию	Допустимые значения
1	id [id]	INT	1	от 1 до 2147483647
2	measurement_id [Id измерения]	INT	1	от 1 до 2147483647
3	pos_x [Позиция по X]	INT	-	от 1 до 1600
4	pos_y [Позиция по Y]	INT	-	от 1 до 1200
5	temperature [Температура]	DECIMAL(6,2)	-	от -9999,99 до 9999,99

В таблице measurements в качестве индекса используется первичный ключ по полю id, который является уникальным и сортируется по возрастанию.

2.3 Проектирование файлов данных

Результатом работы экспорта должен быть файл в формате *.csv и *.docx. Во время работы системы создаются дополнительные файлы, которые сохраняются в оперативную память.

В файлах формата *.csv хранятся данные измерения температуры, взятые с базы данных.

Файлы формата *.docx являются отчётами с маркированными изображениями, самой высокой и самой низкой зафиксированной температурой.

2.4 Организация сбора, передачи, обработки и выдачи информации

Сбор исходной информации выполняется автоматически. Программное обеспечение отправляет HTTP GET-запрос на модуль ESP32-CAM для получения изображения. Данные о температуре получаются с подсистемы в рамках другой ВКР.

Передача изображений с камеры осуществляется по Wi-Fi. Точка доступа создаётся модулем ESP32-CAM.

Выдача информации осуществляется на экран монитора, а также в формате *.docx и *.csv.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		26

3 АЛГОРИТМИЧЕСКОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ

3.1 Алгоритм наложения температурной информации на изображение и сохранения в базу данных

Общая характеристика: автоматическое наложение на оптическое изображение тепловой карты исходя из координат точек с измеренной температурой, а также последующее сохранение модифицированного пути до изображения в базе данных в виде строки.

Используемые данные: изображение с камеры, температура, координаты точки измерения.

Математическое описание: отсутствует.

Результаты выполнения: промаркированное изображение.

Блок-схема алгоритма представлена на рисунке 3.1.

3.2 Алгоритм экспорта в файл в формате CSV

Общая характеристика: создание файла *.csv с данными по сессии с сортировкой по номеру измерения.

Используемые данные: данные из БД.

Результаты выполнения: файл *.csv.

Математическое описание: отсутствует.

Блок-схема алгоритма представлена на рисунке 3.2.

3.3 Алгоритм экспорта в файл в формате DOCX

Общая характеристика: создание файла *.docx с максимальной, минимальной измеренными температурами и изображением с тепловой картой.

Используемые данные: данные из БД, маркированное изображение.

Результаты выполнения: файл *.docx.

Математическое описание: отсутствует.

Блок-схема алгоритма представлена на рисунке 3.3.

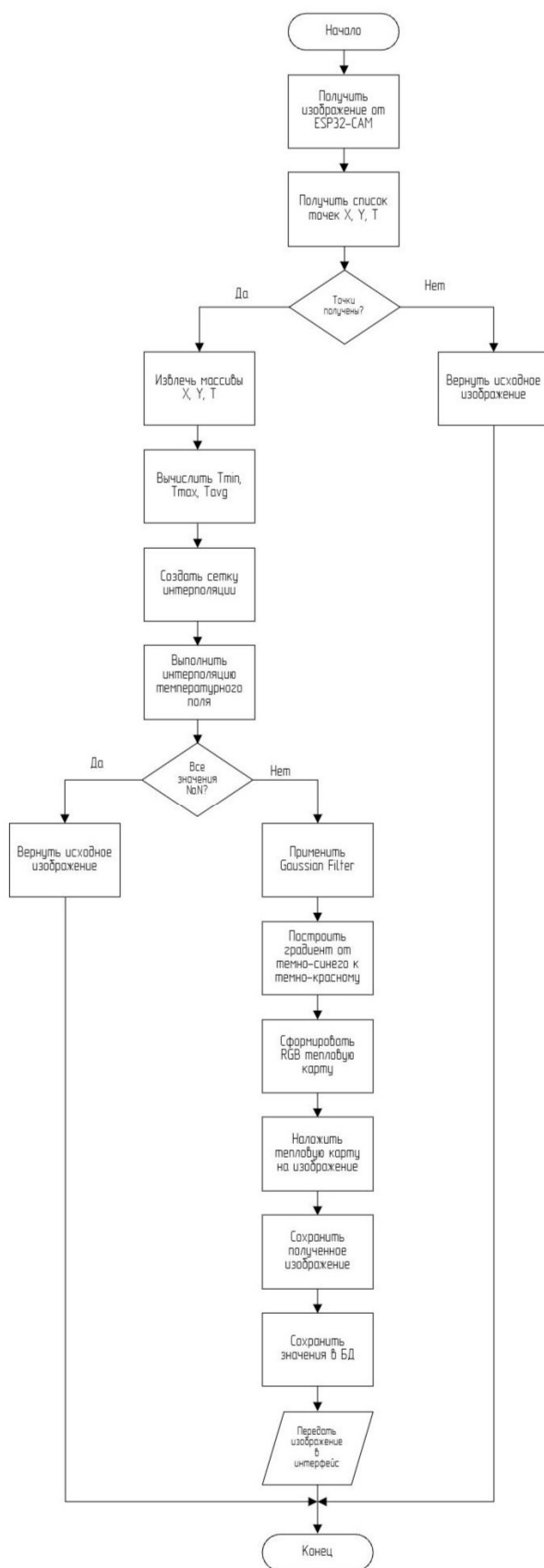


Рисунок 3.1 – Блок-схема алгоритма наложения температурной информации на изображение и сохранения в базу данных

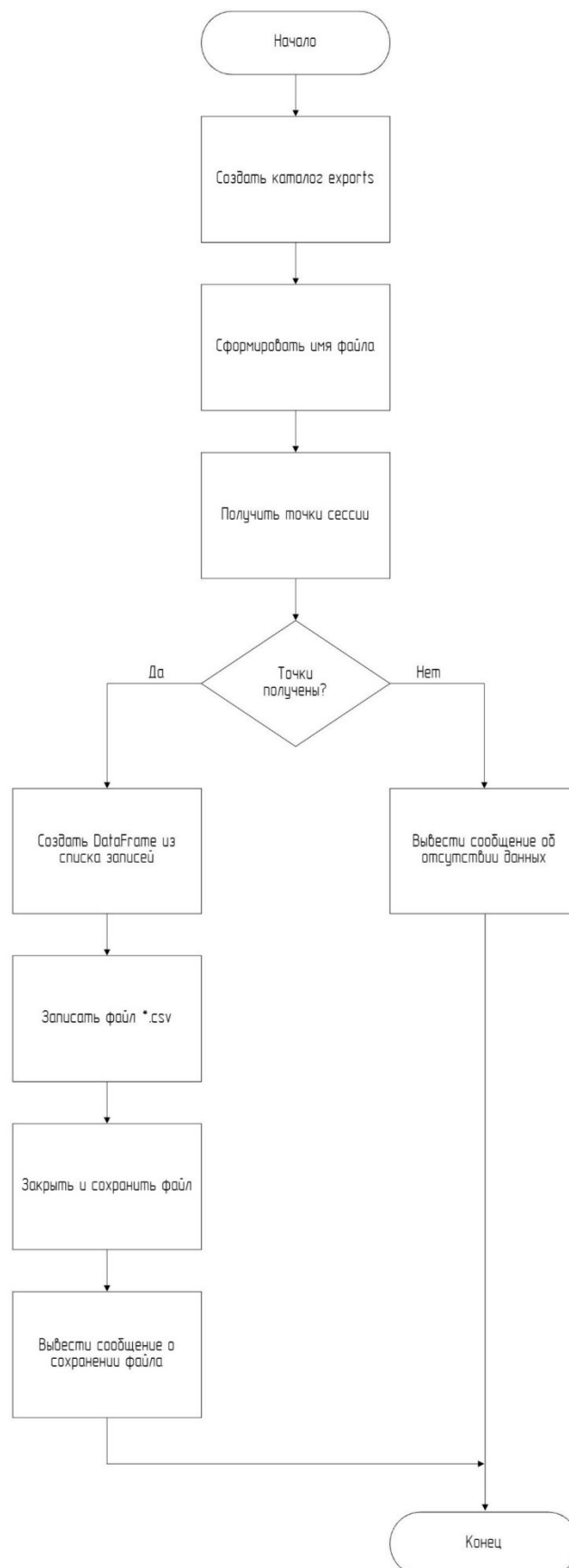


Рисунок 3.2 – Блок-схема алгоритма экспорта в файл в формате CSV

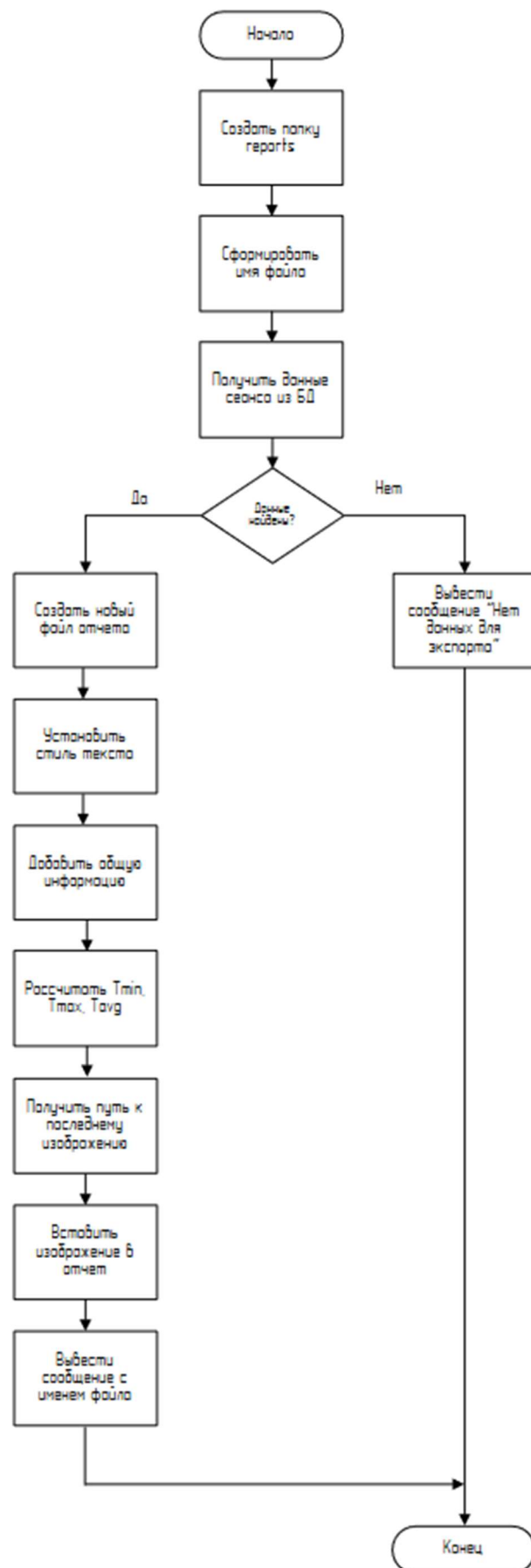


Рисунок 3.3 – Блок-схема алгоритма экспорта в файл в формате DOCX

4 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПОДСИСТЕМЫ

4.1 Структура программного обеспечения и функции его компонентов

ОС – Windows 10.

Инструментальное средство разработки – Visual Studio Code (VSCode).

Язык программирования – Python 3.10.

4.2 Выбор компонентов программного обеспечения

4.2.1 Операционная система

Согласно техническому заданию, подсистема должна быть платформонезависимой. Таким образом, для работы АС подходит большинство современных операционных сред, в том числе бесплатных.

Рассмотрим ОС Ubuntu 16.04 и ОС Windows 10. Обе операционные системы позволяют работать с языком Python, также позволяют установить систему управления базами данных для корректной работы программного обеспечения. Однако с точки зрения разработки целесообразнее использовать операционную систему Windows 10 из-за следующих факторов:

- наличие мощных инструментальных средств разработки и отладки;
- наличие опыта разработки программ на данной операционной системе;
- наличие удобных средств администрирования баз данных.

4.2.2 Инструментальное средство разработки и язык программирования

В качестве языка программирования выбран Python 3.10+ по следующим причинам:

- широкая экосистема библиотек для обработки изображений;
- кроссплатформенность;
- простота и скорость разработки;
- открытость и бесплатность.

Для разработки графического интерфейса использована библиотека Tkinter, которая входит в стандартную установку Python. Альтернативы (PyQt) более сложны в лицензировании и требуют дополнительных зависимостей, однако при необходимости могут быть применены. В данной работе Tkinter достаточен для реализации требуемого функционала: отображение изображений, кнопок управления.

В качестве инструментального средства разработки выбрана среда Visual Studio Code (VSCode) – бесплатный, кроссплатформенный редактор с поддержкой Python, отладкой, интеграцией с Git и большим количеством расширений. Альтернативы (Python IDLE, Notepad++) уступают в функциональности.

4.2.3 Вспомогательное программное обеспечение

Для функционирования системы необходимо СУБД MySQL (версия 8.0) – может быть установлена на той же машине, что и основное приложение, либо на отдельном сервере.

Установка всех зависимостей автоматизирована через файл requirements.txt.

4.3 Разработка прикладного программного обеспечения

4.3.1 Структура прикладного программного обеспечения

Подсистема имеет следующую структуру, представленную в таблице 4.1.

Таблица 4.1 – Структура разработанной подсистемы

№	Название модуля	Описание
1	Модуль сбора данных	Предназначен для получения данных с подсистемы в рамках второй работы ВКР и изображений с модуля ESP32-CAM
2	Модуль визуализации и разметки	Предназначен для наложения тепловой карты на полученное изображение
3	Модуль экспорта	Предназначен для создания файлов формата *.csv с данными сессии
4	Модуль создания отчёта	Предназначен для формирования отчёта с изображением и данными измерения

4.3.1.1 Модуль сбора данных

Спецификация модуля представлена в таблице 4.2.

Таблица 4.2 – Спецификация модуля сбора данных

№	Название	Описание
Классы		
1	class ESP32Client	Содержит методы для подключения подсистемы к ESP32-CAM
2	class ImageProcessor	Содержит метод для преобразования изображения для дальнейшей работы

4.3.1.2 Модуль визуализации и разметки

Спецификация модуля представлена в таблице 4.3.

Таблица 4.3 – Спецификация модуля визуализации и разметки

№	Название	Описание
Классы		
1	class MonitoringThread	Содержит методы для наложения тепловой карты на изображение
2	class HeatMapBuilder	Содержит методы для создания и визуализации тепловой карты исходя из входных данных
3	class ThermalMonitorApp	Содержит методы для отрисовки графического интерфейса внутри сессии и кнопок на нём
4	class SessionSelector	Содержит методы для отрисовки графического интерфейса с выбором о создании или уже существующей сессии

4.3.1.3 Модуль экспорта

Спецификация модуля представлена в таблице 4.4.

Таблица 4.4 – Спецификация модуля экспорта

№	Название	Описание
Классы		
1	class CSVExporter	Содержит метод для создания файла *.csv с данными сессии измерения

4.3.1.4 Модуль создания отчётов

Спецификация модуля представлена в таблице 4.5.

Таблица 4.5 – Спецификация модуля создания отчётов

№	Название	Описание
Классы		
1	class ReportGenerator	Содержит метод для создания отчёта в формате *.docx с изображением, средней, максимальной и минимальной температурой за сеанс измерения

4.4 Особенности реализации, эксплуатации и сопровождения подсистемы

Особенности реализации:

– Все настройки (адрес БД, порт и т.д.) вынесены в конфигурационный файл settings.json, что упрощает перенос системы на другое оборудование.

– Логирование ошибок и информационных сообщений ведётся как в файл (logs/app.log), так и выводится в консоль (при отладке). Формат лога: [YYYY-MM-DD HH:MM:SS] УРОВЕНЬ: сообщение.

– Для обеспечения лицензионной чистоты используются только библиотеки с открытыми лицензиями (MIT, BSD, PSF).

Эксплуатация:

– Система рассчитана на работу в лабораторных и цеховых условиях при температуре окружающей среды от +10 до +35 °С. Компьютер оператора должен быть защищён от пыли и влаги.

– В случае потери связи с модулем ESP32-CAM подсистема продолжает работу, ожидая новые запросы. При восстановлении связи приём данных возобновляется автоматически.

Сопровождение:

- Исходный код полностью открыт, документирован и может быть модифицирован силами штатного программиста предприятия.
- Для обновления системы достаточно заменить файлы приложения и при необходимости выполнить миграцию БД (при изменении структуры таблиц).

4.5 Руководство пользователя

4.5.1 Требования к условиям эксплуатации

- Аппаратные требования: процессор Intel Core i3 или аналогичный, 4 ГБ ОЗУ, от 1 ГБ свободного места на жёстком диске, сетевой адаптер Wi-Fi, дисплей с разрешением не менее 1280×720.
- Программные требования: ОС Windows 10/11 или Linux (Ubuntu 22.04+, ALT Linux), установленный Python 3.10+, MySQL 8.0, браузер (для просмотра справки, опционально).
- Навыки пользователя: базовое владение ПК, умение запускать приложения, работать с файловой системой.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		37

4.5.2 Инсталляция и настройка

1. Установить MySQL, создать базу данных thermal_monitor и пользователя с правами на создание таблиц, запись и чтение.
2. Выполнить скрипт создания таблиц, который прилагается вместе с файлами подсистемы.
3. Установить Python 3.10 и все зависимости командой `pip install -r requirements.txt`.
4. Запустить приложение `python main.py`. При первом запуске автоматически создаётся конфигурационный файл `settings.json`, который необходимо отредактировать, указав параметры подключения к БД (хост, порт, пользователь, пароль).

4.5.3 Порядок и особенности работы

Запуск системы: после запуска `main.py` открывается окно «Выбор сессии» с кнопками «Создать новую сессию» и «Открыть» (рисунок 4.1). После создания новой сессии, открывается окно с температурой и кнопками «Подключиться», «Старт», «Стоп», «Экспорт в CSV», «Создать отчёт DOCX» и «Назад» (рисунок 4.2). Подсистема принимает изображения после нажатия кнопки «Старт»; принятые кадры автоматически обрабатываются и сохраняются. Пользователь может наблюдать последнее изображение с наложенной тепловой картой (рисунок 4.3).

Экспорт данных: нажать кнопку «Экспорт в CSV». Система сформирует CSV-файл в папке `exports`. Выводится сообщение с папкой и названием файла (рисунок 4.4).

Создание отчета: нажать кнопку «Создать отчёт DOCX». Система сформирует отчет в папке `reports`. Выводится сообщение с папкой и названием файла (рисунок 4.5).

Остановка системы: остановить измерения кнопкой «Стоп», закрыть главное окно GUI – приложение завершит работу, корректно закрыв соединение с БД и остановив соединение с модулем ESP32-CAM.

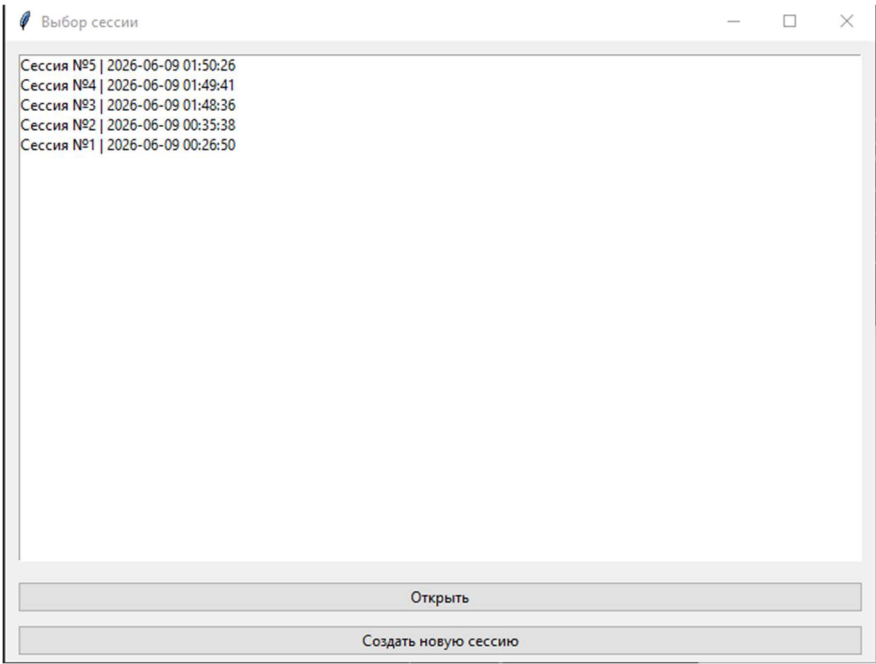


Рисунок 4.1 – Окно «Выбор сессии»



Рисунок 4.2 – Созданная новая сессия



Рисунок 4.3 – Сессия с наложенной тепловой картой

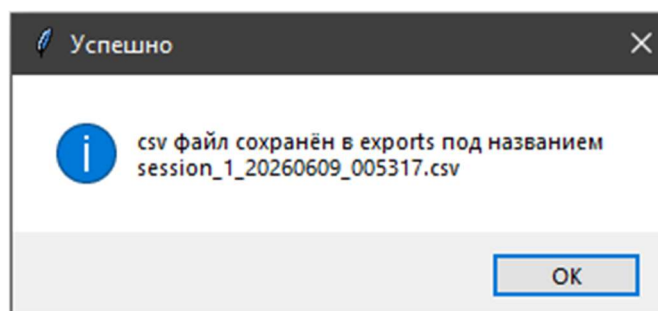


Рисунок 4.4 – Успешный экспорт

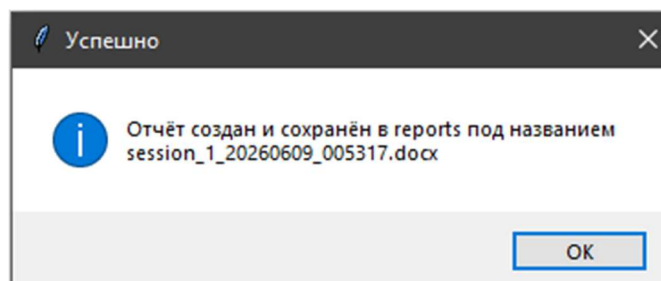


Рисунок 4.5 – Успешное создание отчета

4.5.4 Исключительные ситуации и их обработка

Таблица 4.6 – Исключительные ситуации

Ситуация	Сообщение пользователю	Действие системы
Отсутствие соединения с БД при запуске	«Ошибка подключения к базе данных! Проверьте запущен ли у вас MySQL или файл настроек settings.json!»	Приложение завершает работу с кодом ошибки
Потеря соединения с БД во время работы	«Потеряно соединение с БД, попытка переподключения...»	Система пытается переподключиться каждые 30 с; если успешно – продолжает работу
Некорректный GET-запрос (нет изображения)	Лог «Некорректный запрос: отсутствует поле image»	Возврат HTTP 400, кадр не сохраняется
Повреждённое изображение (Pillow не может декодировать)	Лог «Ошибка декодирования JPEG»	Кадр пропускается, система продолжает работу
Ошибка вставки в БД (например, дубликат первичного ключа, но такого не может быть)	Лог «Ошибка вставки записи: <текст ошибки>»	Кадр не сохраняется, запись в лог
Выход за пределы допустимых значений температуры или координат	Предупреждение в лог	Значения всё равно сохраняются, но пользователь может увидеть нештатные данные

Все критические ошибки выводятся в лог-файл и в специальное окно уведомлений (всплывающее сообщение в GUI). Система спроектирована таким образом, что отказ одного кадра не влияет на общую работоспособность.

5 ТЕСТИРОВАНИЕ СИСТЕМЫ

5.1 Условия и порядок тестирования

Объектом тестирования является прикладное программное обеспечение – десктопное приложение, предназначенное для приёма, обработки, хранения и экспорта инфракрасного мониторинга температуры объекта.

Методом тестирования была выбрана модель черного ящика.

Общий порядок тестирования:

- снимаются входные данные;
- снимаются выходные данные.

5.2 Исходные данные для контрольных примеров

5.2.1 Описание ситуаций для элементов системы

Для тестирования корректности наложения тепловой карты сгенерируем случайную тепловую карту с температурами с предельными значениями датчика MLX90614. Для тестирования работоспособности авторизации в базу данных пробуем изменить settings.json с полями mysql_user на случайную строку (неправильный), а потом на root (правильный). Для тестирования работоспособности модуля экспорта вручную запишем в базу данных случайный набор точек, температур и временных меток.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		42

5.3 Результаты тестирования

Корректная работа описанных выше исходных данных представлена в п.4.5.3 Руководства пользователя. Соответственно в остальных случаях обращаться к п.4.5.4.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
						43
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		

6 ЭКОНОМИЧЕСКИЙ РАЗДЕЛ

6.1 Расчёт показателей трудоёмкости для программного продукта

Величина параметра трудоёмкости для разрабатываемого программного продукта состоит из суммы значений трудоёмкости для каждого этапа разработки и рассчитывается по формуле:

$$T_{об} = \sum_{i=1}^n t_i, \quad (1)$$

где $T_{об}$ – общая трудоёмкость разработки программного продукта;

t_i – трудоёмкость работ на i -й стадии разработки;

n – общее количество этапов разработки.

Таблица 6.1 – Поэтапная и общая оценка трудоёмкости

Этап работы	Вид работы	Трудоёмкость (чел. * час)
Формирование требований к системе	Исследование объекта предметной области Обоснование необходимости создания системы Анализ требований к системе	20
Техническое задание	Разработка и утверждение технического задания на систему	30
Разработка системы	Разработка системы на языке программирования	260
Тестирование системы	Проведение тестирования разработанной системы на тестовых данных Устранение ошибок	80
Рабочая документация	Разработка рабочей документации на систему	50
Общая нагрузка		440

6.2 Расчёт затрат на материальные ресурсы

Расчёт затрат на материальные ресурсы Z_m производится по формуле:

$$Z_m = \sum_{i=1}^n P_i \times C_i, \quad (2)$$

где P_i – расход i -го вида материального ресурса, натуральные единицы;

C_i – цена за единицу i -го вида материального ресурса;

i – вид материального ресурса;

n – общее количество всех видов материальных ресурсов.

Результаты расчетов затрат на материальные ресурсы приведены в таблице 6.2.

Таблица 6.2 – Затраты на материальные ресурсы

Наименование	Единица измерения	Количество израсходованного материала	Цена за единицу, руб.	Сумма, руб.
Компьютер	шт	1	100 000	100 000
Монитор	шт	1	10 000	10 000
Модуль ESP32-CAM с камерой OV3660	шт	1	797	797
Кабель Micro-USB	шт	1	391	391
Итого				111 188

При разработке системы не возникло необходимости в использовании расходных материалов.

Общая сумма затрат на электроэнергию $Z_э$ рассчитывается по формуле:

$$Z_э = \sum_{i=1}^n M_i \times T_i \times Ц, \quad (3)$$

где M_i – паспортная мощность i -го электрооборудования, кВт;

T_i – время работы i -го оборудования за весь период разработки, ч;

$Ц$ – тариф электроэнергии, руб./кВт×ч;

i – вид электрооборудования;

n – количество электрооборудования.

Необходимые расчёты затрат на электроэнергию приведены в таблице 6.3.

Таблица 6.3 – Затраты на электроэнергию

Наименование	Паспортная мощность, кВт	Время работы оборудования	Тариф электроэнергии, руб./кВт×ч	Сумма, руб.
Компьютер	0,70	440	5,64	2 481,6
Монитор	0,20	440	5,64	2 481,6
Освещение	0,04	200	5,64	1 128,0
Итого				6 091,2

6.3 Расчет затрат на разработку системы

Определение затрат на разработку производится путем составления соответствующей сметы, которая включает следующие статьи:

- затраты на оплату труда;
- отчисления на социальные нужды;
- амортизация основных фондов.

6.4 Расчет затрат на оплату труда

Общая сумма затрат на оплату труда $Z_{тр}$ определяется по формуле:

$$Z_{тр} = \sum_{i=1}^n ЧС_i \times T_i, \quad (4)$$

где $ЧС_i$ – часовая ставка i -го работника, руб.;

T_i – время на разработку системы, ч;

i – порядковый номер работника;

n – количество работников.

Общая сумма затрат на оплату труда $Z_{тр}$ определяется по формуле:

$$ЧС_i = \frac{ЗП_i}{ФРВ_i}, \quad (5)$$

где $ЗП_i$ – среднемесячная заработная плата разработчика системы, руб.;

$ФРВ_i$ – среднемесячный фонд рабочего времени.

Стоимость одного часа программиста равна 300 руб.

Общая сумма затрат на оплату труда равна:

$440 \text{ ч.} \times 300 \text{ руб.} = 132\,000 \text{ руб.}$

6.5 Расчет отчислений на социальные нужды

Данные об отчислениях на социальные нужды представлены в таблице 6.4.

Таблица 6.4 – Отчисления на социальные нужды

Вид	Начислено заработной платы, руб.	Отчисления %	Сумма, руб.
Фонд социального страхования РФ	132 000	2,9	3828
Фонд обязательного медицинского страхования		5,1	6732
Пенсионный фонд РФ		22	29040
Итого			39600

6.6 Себестоимость проекта

В таблице 6.5 представлен расчет себестоимости проекта.

Таблица 6.5 – Себестоимость проекта

Статьи затрат	Сумма, руб.
Затраты на материальные ресурсы	111 188
Затраты на оплату труда	132 000
Отчисление на социальные нужды	39 600
Итого по смете	282 788

6.7 Расчет плановой прибыли

Прибыль Π рассчитывается по формуле:

$$\Pi = \frac{C_{\text{пол}} \times P_{\text{н}}}{100}, \quad (6)$$

где $C_{\text{пол}}$ – полная себестоимость, руб.;

$P_{\text{н}}$ – норма рентабельности.

При норме рентабельности 50% плановая прибыль составит:

$$\Pi = 282\,788 \times 50 \% / 100 \% = 141\,394 \text{ руб.}$$

Полная стоимость проекта $C_{\text{пр}}$ определяется как сумма себестоимости проекта и прибыли:

$$C_{\text{пол}} = 282\,788 + 141\,394 = 424\,182 \text{ руб.}$$

С учетом налога на прибыль 20 % доход составит:

$$141\,394 - 141\,394 \times 0,2 = 113\,116 \text{ руб.}$$

6.8 Определение экономической эффективности проекта

Определение экономической эффективности проекта проводилась по методу расчета экономического эффекта от прибыли по следующей формуле:

$$\mathcal{E}_{\mathcal{E}} = \frac{\Pi}{C_{\text{пол}}} \times 100 \%, \quad (7)$$

где $\mathcal{E}_{\mathcal{E}}$ – экономический эффект, %;

$C_{\text{пол}}$ – себестоимость, руб.;

Π – прибыль (за вычетом налога на прибыль), руб.

Экономический эффект равен:

$$\mathcal{E}_{\mathcal{E}} = (113\,116 / 424\,182) \times 100 \% = 27 \.%$$

6.9 Выводы по технико-экономическому анализу и обоснованию проекта разработки

В результате проведения технико-экономического анализа проекта были рассчитаны несколько показателей, с помощью которых было получено технико-экономическое обоснование разработки. Себестоимость проекта составила 321 600 руб. Полная стоимость проекта составила 482 400 руб. Доход от внедрения системы - 128 640 руб. Экономический эффект составил 27%.

Исходя из показателей, разработка системы является эффективной и принесет прибыль.

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		50

ЗАКЛЮЧЕНИЕ

В данной выпускной квалификационной работе разработано программное обеспечение для управления камерой и обработки данных в системе инфракрасного мониторинга температуры объекта.

В ходе выполнения работы были проанализированы существующие программные решения для тепловизионного контроля, сформировано техническое задание и определены требования к разрабатываемой системе. Выполнено проектирование информационного обеспечения, разработана структура базы данных для хранения результатов измерений, а также определены форматы входных и выходных данных.

Разработанное программное обеспечение обеспечивает подключение к модулю ESP32-CAM, приём изображений и температурных данных, автоматическое наложение температурной информации на изображения, сохранение результатов измерений в базе данных MySQL, а также экспорт данных в форматы CSV и DOCX. Для реализации системы использованы язык программирования Python, библиотека Tkinter для построения графического интерфейса пользователя и библиотека Pillow для обработки изображений.

В процессе работы были разработаны алгоритмы визуализации температурных данных, сохранения результатов измерений и формирования отчётов. Проведённое тестирование подтвердило корректность работы всех основных функций системы, включая получение данных, отображение тепловой карты, экспорт результатов и создание отчётной документации.

Разработанная система позволяет автоматизировать процесс теплового контроля электронных компонентов, печатных плат и готовых изделий, снизить влияние человеческого фактора при проведении измерений и обеспечить удобное хранение результатов для последующего анализа. Использование открытых технологий и модульной архитектуры

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		51

обеспечивает возможность дальнейшего расширения функциональности и интеграции системы в более крупные комплексы мониторинга и диагностики.

Экономический анализ показал целесообразность разработки и внедрения программного обеспечения. Полученные результаты подтверждают достижение поставленной цели и решение всех задач, сформулированных в выпускной квалификационной работе.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Введ. 1992-01-01. – М. : Стандартинформ, 2010. – 26 с.

2. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Введ. 1990-01-01. – М. : Стандартинформ, 2009. – 11 с.

3. Pillow (PIL) Documentation [Электронный ресурс] / Python Imaging Library. – Режим доступа: <https://python-pillow.org/> (дата обращения: 01.06.2026).

4. Tkinter Documentation [Электронный ресурс] / Python Software Foundation. – Режим доступа: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 01.06.2026).

5. Python CSV Module Documentation [Электронный ресурс] / Python Software Foundation. – Режим доступа: <https://docs.python.org/3/library/csv.html> (дата обращения: 01.06.2026).

6. http.server – HTTP servers [Электронный ресурс] / Python Software Foundation. – Режим доступа: <https://docs.python.org/3/library/http.server.html> (дата обращения: 01.06.2026).

7. MLX90614 Datasheet [Электронный ресурс] / Melexis Microelectronic Systems. – Режим доступа: <https://www.melexis.com/en/documents/documentation/datasheets/datasheet-mlx90614> (дата обращения: 01.06.2026).

8. Santos, R. ESP32-CAM HTTP Post Images/Photos to Server [Электронный ресурс]. – Режим доступа: <https://randomnerdtutorials.com/esp32-cam-http-post-image-server/> (дата обращения: 01.06.2026).

9. Adafruit CircuitPython MLX90614 Library [Электронный ресурс] / Adafruit Industries. – Режим доступа: https://github.com/adafruit/Adafruit_CircuitPython_MLX90614 (дата обращения: 01.06.2026).

10. Оценка трудоемкости разработки программного продукта : метод. указания / сост. Н. И. Шанченко. – Ульяновск : УлГТУ, 2015. – 21 с.

11. Обработка изображений на Python с использованием библиотеки Pillow [Электронный ресурс] / Skillfactory. – Режим доступа: <https://blog.skillfactory.ru/python-pillow/> (дата обращения: 01.06.2026).

12. Библиотека Tkinter в Python [Электронный ресурс] / Selectel. – Режим доступа: <https://selectel.ru/blog/tutorials/tkinter-python/> (дата обращения: 01.06.2026).

13. Эффективная обработка CSV-файлов с помощью Python [Электронный ресурс] / TutKit.com. – Режим доступа: <https://www.tutkit.com/> (дата обращения: 01.06.2026).

14. Разработка графического интерфейса на Tkinter [Электронный ресурс] / Real Python. – Режим доступа: <https://realpython.com/python-gui-tkinter/> (дата обращения: 01.06.2026).

15. Бесконтактный датчик температуры MLX90614 [Электронный ресурс] / RobotClass. – Режим доступа: <https://robotclass.ru/tutorials/sensor-mlx90614/> (дата обращения: 01.06.2026).

16. Работа с CSV-файлами на Python [Электронный ресурс] / LabEx. – Режим доступа: <https://labex.io/> (дата обращения: 01.06.2026).

17. Гонсалес, Р. Ц. Цифровая обработка изображений / Р. Ц. Гонсалес, Р. Э. Вудс. – 3-е изд. – М. : Техносфера, 2012. – 1104 с. – ISBN 978-5-94836-331-8.

18. Вавилов, В. П. Инфракрасная термография и тепловой контроль / В. П. Вавилов, А. Г. Климов. – М. : Спектр, 2011. – 380 с. – ISBN 978-5-4442-0069-6.

19. Свейгарт, Э. Automate the Boring Stuff with Python / Э. Свейгарт. – 2nd ed. – San Francisco : No Starch Press, 2019. – 592 p. – ISBN 978-1-59327-992-9.

20. Лутц, М. Изучаем Python / М. Лутц. – 5-е изд. – СПб. : БХВ-Петербург, 2019. – 1296 с. – ISBN 978-5-9775-3949-4.

21. MLX90614. Бесконтактный инфракрасный датчик температуры : руководство по применению / Melexis Microelectronic Systems. – Ieper : Melexis, 2021. – 52 p.

					<i>ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ</i>	<i>Лист</i>
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		55

ПРИЛОЖЕНИЕ А

(обязательное)

Исходные тексты программы

main.py

```
import json
from session_selector import SessionSelector

with open("config/settings.json", "r", encoding="utf-8") as f:
    settings = json.load(f)

if __name__ == "__main__":
    SessionSelector(settings).run()
```

gui.py

```
import tkinter as tk
import json
import os
from tkinter import ttk, messagebox
from PIL import ImageTk, Image
from database import DatabaseManager
from monitoring import MonitoringThread
from exporter import CSVExporter
from report_generator import ReportGenerator
from esp32_client import ESP32Client
from datetime import datetime

stamp = datetime.now().strftime("%Y%m%d_%H%M%S")

class ThermalMonitorApp:

    def __init__(self, settings, session_id, is_new_session=False):

        self.settings=settings
        self.session_id=session_id
        self.is_new_session = is_new_session

        self.root=tk.Tk()
        self.root.title(f"Мониторинг - Сессия №{session_id}")
        self.root.geometry("1200x1000")

        self.db=DatabaseManager(settings)

        self.connected = False
        self.monitor_thread = None

        self.image_label=tk.Label(self.root)
        self.image_label.pack(expand=True)

        self.info_frame = ttk.Frame(self.root)
        self.info_frame.pack(fill="x", padx=10, pady=5)

        self.lbl_points = ttk.Label(self.info_frame, text="Точек: -")
        self.lbl_points.pack(anchor="w")

        self.lbl_min = ttk.Label(self.info_frame, text="Мин. температура: -")
        self.lbl_min.pack(anchor="w")

        self.lbl_max = ttk.Label(self.info_frame, text="Макс. температура: -")
        self.lbl_max.pack(anchor="w")

        self.lbl_avg = ttk.Label(self.info_frame, text="Средняя температура: -")
```

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист 56
Изм.	Лист	№ докум.	Подпись	Дата		


```

self.lbl_avg.pack(anchor="w")

if self.is_new_session:
    self.connect_btn = ttk.Button(self.root, text="Подключиться",
command=self.connect)
    self.connect_btn.pack()

    self.start_btn = ttk.Button(self.root, text="Старт",
command=self.start_monitoring)
    self.start_btn.pack()

    self.stop_btn = ttk.Button(self.root, text="Стон", command=self.stop_monitoring)
    self.stop_btn.pack()

    ttk.Button(self.root, text="CSV", command=self.export_csv).pack()
    ttk.Button(self.root, text="DOCX", command=self.export_docx).pack()
    ttk.Button(self.root, text="Назад", command=self.back_to_sessions).pack()

if not self.is_new_session:
    self.load_last_image()

def update_monitor( self, image, stats, ts):

    self.root.after(0, lambda: self._update_ui(image, stats))

def _u(self, image):
    #print("GUI получил изображение", image.size)

    image.thumbnail((900, 600))
    p=ImageTk.PhotoImage(image)
    self.current_image=p
    self.image_label.configure(image=p)

def _update_ui(self, image, stats):

    self._u(image)

    self.lbl_points.configure( text=f"Точек: {stats['points']}")

    self.lbl_min.configure(text=(
        f"Мин. температура: "
        f"{stats['tmin']:.2f} °C"))

    self.lbl_max.configure(text=(
        f"Макс. температура: "
        f"{stats['tmax']:.2f} °C"))

    self.lbl_avg.configure(text=(
        f"Средняя температура: "
        f"{stats['tavg']:.2f} °C"))

def load_last_image(self):

    path = self.db.get_last_image(self.session_id)

    if not path:
        return

    try:
        image = Image.open(path)
        self._u(image)

    except Exception as e:

```

```

        print(f"Ошибка загрузки изображения: {e}")

def connect(self):
    try:
        client = ESP32Client(self.settings["esp32_ip"])

        client.get_frame()
        self.connected = True
        print("ESP32 подключена")

    except Exception as e:
        self.connected = False
        print("Ошибка подключения:", e)

def start_monitoring(self):

    if not self.connected:
        print("Сначала выполните подключение")
        return

    if (self.monitor_thread and self.monitor_thread.is_alive()):
        return

    self.monitor_thread = MonitoringThread(self.session_id, self.settings["esp32_ip"],
self.db, self.update_monitor)

    self.monitor_thread.start()

def stop_monitoring(self):

    if self.monitor_thread:

        self.monitor_thread.stop()
        self.monitor_thread = None

def export_csv(self):
    os.makedirs("exports", exist_ok=True)
    filename = (f"exports/" + f"session_{self.session_id}_{stamp}.csv")

    CSVExporter.export(self.db.get_session_points(self.session_id), filename)

    messagebox.showinfo(title="Успешно", message=f"csv файл сохранён в exports под
названием session_{self.session_id}_{stamp}.csv")

def export_docx(self):
    os.makedirs("reports", exist_ok=True)
    filename = (f"reports/" + f"session_{self.session_id}_{stamp}.docx")

    ReportGenerator.generate(filename, self.session_id,
        self.db.get_measurements(self.session_id),
        self.db.get_session_points(self.session_id),
        self.db.get_last_image(self.session_id))

    messagebox.showinfo(title="Успешно", message=f"Отчёт создан и сохранён в reports под
названием session_{self.session_id}_{stamp}.docx")

def back_to_sessions(self):

    if self.monitor_thread:
        self.monitor_thread.stop()
    self.root.destroy()

```

```
from session_selector import SessionSelector
SessionSelector(self.settings).run()
```

```
def run(self):
    self.root.mainloop()
```

monitoring.py

```
import os, random, threading
from datetime import datetime
from esp32_client import ESP32Client
from image_processor import ImageProcessor
from logger import logger
import traceback
```

```
class MonitoringThread(threading.Thread):
```

```
    def __init__(self, session_id, ip, db, cb):
        super().__init__(daemon=True)
        self.session_id=session_id
        self.ip=ip
        self.db=db
        self.cb=cb
        self.running=False
        self.stop_event=threading.Event()
```

```
    def stop(self):
        self.running=False
        self.stop_event.set()
```

```
    def run(self):
```

```
        self.running = True
```

```
        camera = ESP32Client(self.ip)
```

```
        while self.running:
```

```
            try:
```

```
                frame = camera.get_frame()
                ts = datetime.now()
```

```
                points = []
```

```
                for _ in range(300):
```

```
                    points.append(
                        {
                            "x": random.randint(0, 799),
                            "y": random.randint(0, 599),
                            "temp": round(random.uniform(20,90),2)
                        })
```

```
                image = ImageProcessor.load_image(frame)
```

```
                from heatmap import HeatMapBuilder
                (result_image, tmin, tmax, tavg) = HeatMapBuilder.build_overlay(image,points)
```

```
                folder = (f"images/session_{self.session_id}")
```

```
                os.makedirs(folder,exist_ok=True)
```

```
                path = (f"{folder}/{ts.strftime('%Y%m%d_%H%M%S')}.jpg")
```

					BKP-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		59

```

        result_image.convert("RGB").save(path)

        measurement_id = self.db.create_measurement(self.session_id, ts, path)

        self.db.insert_points(measurement_id, points)
        print("Передача изображения в GUI")

        self.cb(result_image,{"tmin": tmin, "tmax": tmax, "tavg": tavg, "points":
len(points)}, ts)
        #print("HeatMap:", result_image, tmin, tmax, tavg)

    except Exception:

        logger.error(
            traceback.format_exc()
        )

    if self.stop_event.wait(5):
        break

```

logging.py

```

import os, logging

os.makedirs("logs", exist_ok=True)

with open("logs/app.log", "w", encoding="utf-8"):
    pass

logging.basicConfig(filename="logs/app.log", level=logging.INFO,
                    format="%(asctime)s | %(levelname)s | %(message)s",
                    encoding="utf-8")

logger = logging.getLogger("thermal_monitor")

```

exporter.py

```

import pandas as pd
class CSVExporter:
    @staticmethod

    def export(records,filename):
        pd.DataFrame(records).to_csv(filename,sep=";",index=False,encoding="utf-8-sig")

```

report_generator.py

```

from docx import Document
from docx.shared import Cm, Pt

class ReportGenerator:

    @staticmethod
    def generate(filename, session_id, records, points, image_path):

        d = Document()

        style = d.styles["Normal"]
        style.font.name = "Times New Roman"
        style.font.size = Pt(12)

        d.add_heading(f"Отчёт по сессии №{session_id}", 1)
        d.add_paragraph(f"Количество кадров: {len(records)}")
        d.add_paragraph(f"Количество точек: {len(points)}")

        if points:

```

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		60

```

temps = [float(p["temperature"]) for p in points]
d.add_paragraph(f"Мин. температура: {min(temps):.2f} °C")
d.add_paragraph(f"Макс. температура: {max(temps):.2f} °C")
d.add_paragraph(f"Средняя температура: {sum(temps)/len(temps):.2f} °C")

if image_path:
    try:
        d.add_picture(image_path, width=Cm(12))

    except Exception:
        pass

d.save(filename)

```

heatmap.py

```

import numpy as np

from PIL import Image

from scipy.interpolate import griddata
from scipy.ndimage import gaussian_filter

class HeatMapBuilder:

    @staticmethod
    def build_heatmap(width, height, points, sigma=3):

        if len(points) < 3:
            return None, None, None, None

        xs = np.array([p["x"] for p in points], dtype=np.float32)
        ys = np.array([p["y"] for p in points], dtype=np.float32)
        temps = np.array([p["temp"] for p in points], dtype=np.float32)

        tmin = float(np.min(temps))
        tmax = float(np.max(temps))
        tavg = float(np.mean(temps))

        grid_y, grid_x = np.mgrid[0:height, 0:width]
        grid = griddata((xs, ys), temps, (grid_x, grid_y), method="cubic", fill_value=np.nan)
        mask = np.isnan(grid)

        if np.all(mask):
            return None, tmin, tmax, tavg

        mean_temp = np.nanmean(grid)
        grid[mask] = mean_temp
        grid = gaussian_filter(grid, sigma=sigma)

        if abs(tmax - tmin) < 0.001:
            norm = np.zeros_like(grid)

        else:
            norm = (grid - tmin) / (tmax - tmin)

        norm = np.clip(norm, 0, 1)

        heatmap = np.zeros((height, width, 4), dtype=np.uint8)

        cold = np.array([0, 0, 139], dtype=np.float32)
        hot = np.array([139, 0, 0], dtype=np.float32)
        rgb = (cold * (1 - norm[:, :, None]) + hot * norm[:, :, None])

        heatmap[:, :, :3] = rgb.astype(np.uint8)

```

```

heatmap[:, :, 3] = 180
heatmap_img = Image.fromarray(heatmap, mode="RGBA" )

return (heatmap_img, tmin, tmax, tavg)

@staticmethod
def overlay(image, heatmap):

    base = image.convert("RGBA")

    return Image.alpha_composite(base, heatmap)

@staticmethod
def build_overlay(image, points):

    width, height = image.size
    (heatmap, tmin, tmax, tavg) = HeatMapBuilder.build_heatmap(width, height, points)

    if heatmap is None:
        return (image, tmin, tmax, tavg)

    result = HeatMapBuilder.overlay(image, heatmap)

    return (result, tmin, tmax, tavg)

```

session_selector.py

```

import tkinter as tk
from tkinter import ttk
from database import DatabaseManager
from gui import ThermalMonitorApp

class SessionSelector:

    def __init__(self, settings):
        self.settings = settings
        self.db = DatabaseManager(settings)

        self.root = tk.Tk()
        self.root.title("Выбор сессии")
        self.root.geometry("700x500")

        self.sessions = tk.Listbox(self.root)
        self.sessions.pack(fill="both", expand=True, padx=10, pady=10)

        ttk.Button(self.root, text="Открыть", command=self.open_session).pack(fill="x",
        padx=10, pady=5)
        ttk.Button(self.root, text="Создать новую сессию",
        command=self.create_session).pack(fill="x", padx=10, pady=5)

        self.load_sessions()

    def load_sessions(self):
        self.sessions.delete(0, tk.END)
        for s in self.db.get_sessions():
            self.sessions.insert(tk.END, f"Сессия №{s['id']} | {s['created_at']}")

    def create_session(self):
        session_id = self.db.create_session()
        self.root.destroy()
        ThermalMonitorApp(self.settings, session_id, is_new_session=True).run()

    def open_session(self):
        sel = self.sessions.curselection()
        if not sel:

```

					ВКР-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		62

```

        return
    session = self.db.get_sessions()[sel[0]]
    self.root.destroy()
    ThermalMonitorApp(self.settings, session["id"], is_new_session=False).run()

def run(self):
    self.root.mainloop()

```

image_processor.py

```

from PIL import Image
from io import BytesIO

class ImageProcessor:

    @staticmethod
    def load_image(frame):
        image = Image.open(BytesIO(frame)).convert("RGB")

        return image

```

database.py

```

import mysql.connector

class DatabaseManager:

    def __init__(self, c):

        self.connection = mysql.connector.connect(
            host=c["mysql_host"],
            port=c["mysql_port"],
            user=c["mysql_user"],
            password=c["mysql_password"],
            database=c["mysql_database"]
        )

        self.cursor = self.connection.cursor(
            dictionary=True
        )

    def create_session(self):

        self.cursor.execute(
            """
            INSERT INTO sessions(created_at)
            VALUES(NOW())
            """
        )

        self.connection.commit()

        return self.cursor.lastrowid

    def get_sessions(self):

        self.cursor.execute(
            """
            SELECT *
            FROM sessions
            ORDER BY id DESC
            """
        )

        return self.cursor.fetchall()

```

					BKP-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист 63
Изм.	Лист	№ докум.	Подпись	Дата		

```

def create_measurement(self, session_id, timestamp, image_path):

    self.cursor.execute(
        """
        INSERT INTO measurements
        (session_id, timestamp, image_path)
        VALUES(%s,%s,%s)
        """,
        (session_id, timestamp, image_path))

    self.connection.commit()

    return self.cursor.lastrowid

def insert_points(self, measurement_id, points):
    rows = []

    for p in points:
        rows.append((measurement_id, p["x"], p["y"], p["temp"]))

    self.cursor.executemany(
        """
        INSERT INTO temperature_points
        (
            measurement_id,
            pos_x,
            pos_y,
            temperature
        )
        VALUES(%s,%s,%s,%s)
        """, rows)

    self.connection.commit()

def get_measurements(self, session_id):

    self.cursor.execute(
        """
        SELECT *
        FROM measurements
        WHERE session_id=%s
        ORDER BY timestamp DESC
        """,
        (session_id,))

    return self.cursor.fetchall()

def get_points(self, measurement_id):

    self.cursor.execute(
        """
        SELECT
            pos_x,
            pos_y,
            temperature
        FROM temperature_points
        WHERE measurement_id=%s
        """, (measurement_id,))

    return self.cursor.fetchall()

def get_last_image(self, session_id):

    self.cursor.execute(
        """
        SELECT image_path
        FROM measurements

```



```

        WHERE session_id=%s
        ORDER BY timestamp DESC
        LIMIT 1
        """, (session_id,))

    row = self.cursor.fetchone()

    return (row["image_path"] if row else None)

def close(self):
    try:
        self.cursor.close()
        self.connection.close()

    except Exception:
        pass

def get_session_points(self, session_id):
    self.cursor.execute(
        """
        SELECT
            m.id AS measurement_id,
            m.timestamp,
            p.pos_x,
            p.pos_y,
            p.temperature
        FROM measurements m
        JOIN temperature_points p
            ON p.measurement_id = m.id
        WHERE m.session_id=%s
        ORDER BY m.timestamp
        """, (session_id,))

    return self.cursor.fetchall()

```

esp32_client.py

```

import requests
from logger import logger

class ESP32Client:
    def __init__(self, ip): self.ip=ip

    def get_frame(self):

        url = f"http://{self.ip}/image"

        r = requests.get(url, timeout=(15))

        logger.error(f"ESP32 response: {r.status_code}")

        return r.content

```

cam_server.ino

```

#include <Arduino.h>
#include <WebServer.h>
#include <WiFi.h>
#include <esp_camera.h>
#include <esp_wifi.h>
#include <soc/rtc_cntl_reg.h>
#include <soc/soc.h>

// wifi
#define WIFI_SSID "ESP32-CAM"

```

					BKP-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		65

```

#define WIFI_PASS "12345678"

// camera pins Ai Thinker
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27

#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

// camera init
bool cameraInit(framesize_t frame_size, pixformat_t pixel_format, int jpeg_quality) {
    esp_wifi_set_ps(WIFI_PS_NONE); // no power save
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // disable brownout

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.frame_size = FRAMESIZE_UXGA;
    config.pixel_format = PIXFORMAT_JPEG; // for streaming
    //config.pixel_format = PIXFORMAT_RGB565; // for face detection/recognition
    config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
    config.fb_location = CAMERA_FB_IN_PSRAM;
    config.jpeg_quality = 12;
    config.fb_count = 1;

    if (config.pixel_format == PIXFORMAT_JPEG) {
        if (psramFound()) {
            config.jpeg_quality = 10;
            config.fb_count = 2;
            config.grab_mode = CAMERA_GRAB_LATEST;
        } else {
            // Limit the frame size when PSRAM is not available
            config.frame_size = FRAMESIZE_SVGA;
            config.fb_location = CAMERA_FB_IN_DRAM;
        }
    } else {
        // Best option for face detection/recognition
        config.frame_size = FRAMESIZE_240X240;
    }
}

```

					BKP-УлГТУ-09.03.02-22/24 10-2026ПЗ	Лист 66
Изм.	Лист	№ докум.	Подпись	Дата		

```

    }

    return esp_camera_init(&config) == ESP_OK;
}

WebServer server(80);

void setup() {
    Serial.begin(115200);
    Serial.println();

    // camera
    if (!cameraInit(FRAMESIZE_VGA, PIXFORMAT_JPEG, 4)) {
        Serial.println("Camera error");
        for (;;);
    }

    // wifi
    WiFi.softAP(WIFI_SSID, WIFI_PASS);

    server.on("/image", []() {
        Serial.println("HTTP request received");

        camera_fb_t* fb = esp_camera_fb_get();
        sensor_t *s = esp_camera_sensor_get();

        if (s->id.PID == OV3660_PID) {
            s->set_vflip(s, 1); // flip it back
            s->set_brightness(s, 1); // up the brightness just a bit
            s->set_saturation(s, -2); // lower the saturation
        }
        if (fb) {
            server.setContentLength(fb->len);
            server.send(200, "image/jpeg", "");
            server.sendContent((const char*)fb->buf, fb->len);
        }
        esp_camera_fb_return(fb);
    });
    server.begin();
}

void loop() {
    server.handleClient();
}

```